

DoctorApp — Daily Report

Junaid Ansari

Shortlisting Intern Tasks • Day 4 Status

1 What did you build today?

I completed the implementation of the full, end-to-end appointment booking flow in the React Native project. Specifically, the following screens and elements were built:

- **DoctorProfileScreen** **COMPLETE** : Extended to implement a stateful custom tab switcher swap (About vs Reviews tabs) without external navigation dependencies. Integrates review cards rendering ratings, comments, dates, and star rating graphics (★/☆).
- **SlotPickerScreen** **COMPLETE** : Renders a horizontal scrollable 7-day selector dynamically calculated using Javascript's Date API. Displays a 3-column time slot grid indicating available (white), selected (blue highlight), and booked (gray strike-through, disabled) slot states.
- **BookingSummaryScreen** **COMPLETE** : Reuses DoctorCard and calculates consultation fees, platform fee (₹50), and total fee. Includes form input for patient visit reason.
- **BookingConfirmationScreen** **COMPLETE** : Success screen with customized graphic checkmark, appointment overview, and dynamic Booking ID generator. Resets navigation stack back to Home.
- **App Logo & Splash/Login Integration** **COMPLETE** : Generated custom medical cross app logo, converted it to proper PNG encoding, set it as the native Android app launcher icon, and added it to the headers of the Splash and Login screens.

2 What did you learn?

I learned a concrete lesson regarding the Android AAPT2 asset compiler: **AAPT2 strictly validates image file signatures matching the file header magic bytes** rather than relying on the file extension. If an AI generator produces a JPEG image but appends a `.png` extension, Android build tasks will crash with the error "failed to read PNG signature". To resolve this without installing Python image libraries, we can programmatically load the file through `.NET`'s `System.Drawing.Bitmap` assembly inside a PowerShell command and save it using standard ImageFormat.Png` encoding to generate a valid PNG header signature.`

3 Where did you get stuck?

I got stuck during the Gradle build compilation step when AAPT2 failed resource compilation because the generated app logo file was internally encoded as a JPEG despite the `.png` extension (signature mismatch). The build failed with `failed to read PNG signature`. This issue took **about 20 minutes** to diagnose (using .NET binary byte reading `[System.IO.File]::ReadAllBytes` to find JPEG magic bytes `255 216 255 224`). I resolved it by running a PowerShell command invoking GDI+ bitmap encoding to convert the file into a true PNG signature on disk, allowing Gradle to compile successfully.

4 What will you do tomorrow?

Tomorrow (Day 5 - Polish & My Profile), I will specifically tackle:

- **My Profile Screen:** Implement a user details page allowing editing of profile info (name, mobile, email) and local profile avatar setup.
- **Local State & AsyncStorage Integration:** Configure local offline storage so that patient details and booking histories (unique Booking IDs and appointment metadata) are persistent and accessible in a mock "My Appointments" panel.
- **Polishing Navigation Transitions:** Smooth out transitions, customize fonts, and ensure consistent color tokens are used on all components.

5 GitHub link

Pushed all updates to the remote repository. The updated link is:

<https://github.com/junaidansari096/Doctor-booking-app.git>

6 AI prompts you used today

The copy-pasted prompts used today (restructured as professional developer instruction sets):

PROMPT 1: APP LOGO & BRANDING GENERATION

Act as a senior UI/UX designer. Generate a high-fidelity, minimalist, modern medical-themed icon representing a pulse line crossing a professional medical cross. Use a clean color palette of turquoise and royal blue, styled on a neutral background, suitable for a mobile application launcher icon.

PROMPT 2: NATIVE ANDROID RESOURCE COMPILER ERROR RESOLUTION

Develop a PowerShell script utilizing .NET's [System.Drawing] assembly to load an image asset file and save it in a true PNG format. The script must overwrite the file's raw binary signature and write standard PNG magic headers to resolve Android

AAPT2 compiler 'failed to read PNG signature' resource validation errors during the release build.

PROMPT 3: STATEFUL DOCTOR PROFILE TAB BAR & REVIEWS FEED

In React Native, extend our existing `DoctorProfileScreen` component by implementing a custom, stateful Tab Switcher (About vs. Reviews) using only native `[useState]` hooks. Under the Reviews tab, render a list of patient review cards with custom star-rating graphics (★/☆), formatted review dates, and patient comments from a structured dummy array.

PROMPT 4: SCROLLABLE 7-DAY DATE & TIME SLOT GRID SELECTOR

Create a stateful `SlotPickerScreen` component in React Native. Implement a horizontal, scrollable 7-day date selector using the JavaScript Date API. Below it, render a 3-column grid of time slots featuring three visual states: 'Available' (selectable, white), 'Booked' (disabled, strike-through, gray), and 'Selected' (highlighted, brand blue). Restrict navigation to the next screen until a slot is selected.

PROMPT 5: BOOKING CHECKOUT SUMMARY & COST CALCULATIONS

Build a `BookingSummaryScreen` in React Native that receives the doctor details, selected date, and selected time slot. Render a detailed cost breakdown displaying a Consultation Fee (e.g., ₹500), a fixed Platform Fee (₹50), and a calculated Total. Include a validated patient visit reason input field and a confirm booking CTA.

PROMPT 6: SUCCESS CONFIRMATION SCREEN & STACK NAVIGATION RESET

Implement a `BookingConfirmationScreen` featuring a success checkmark animation, the booked doctor details, and a randomized booking ID generator (format: 'BK' followed by a 6-digit number). Configure the 'Go to Home' button to reset the React Navigation Native Stack, clearing all intermediate booking screens from history to prevent back-navigation.